

STREAM PROCESSING AND ANALYTICS

(SI-25-DSECLZG556)

ASSIGNMENT 1

GROUP NAME → Group 78 (Problem Statement)

SPA - Assignment - 2 - Streaming Data Solutions
on AWS

CONTRIBUTION TABLE

Sl No.	Name	ID No.	Contribution
1	BHARTI CHAUDHARY	2023DC04011	100%
2	ANSHIO S. V.	2024DA04064	100%
3	HAREESHA BANDARU	2024DA04089	100%

Q1/ You are consulting for a smart manufacturing company aiming to transition from batch analytics to real-time decision automation. The leadership wants to understand how streaming analytics benefits different operational areas. [...]

Solⁿ Based on the AWS whitepaper, the following are five new real-world examples - one for each streaming use case category.

1. Real-Time Monitoring - Continuous monitoring of patient vitals in hospitals

Scenario

Hospitals use IoT-enabled monitors to continuously track patients' heart rate, oxygen levels, ECG and blood pressure in real time

Use of AWS Streaming Analytics

✓ Vitals from bedside devices stream into AWS IoT Core
↓
Kinesis Data Streams

✓ Kinesis Data Analytics (Flink SQL) evaluates vitals against medical thresholds

✓ Processed values are stored in DynamoDB/SS, and live graphs update on doctors' dashboards.

Justification

This fits real-time monitoring because patient vital signs must be observed continuously with instant visibility to detect changes as they occur.

2. Real-Time Notification - Sales Representative Triggered by Consumer Online Activity

Scenario

A company monitors online behaviour such as product page visits, downloads and cart addition. When a user shows buying intent, sales representatives receive an immediate alert.

Use of AWS Streaming Analytics

- > User activity ~~flow~~ events flow into Kinesis Data Streams.
- > AWS Lambda identifies high-intent behaviours (multiple visits, high engagements).
- > Matching events trigger SNS/Email/SMS notification to sales teams.

Justification

This is real-time notification because alerts must be pushed instantly to the sales team so they can engage the lead at the optimal moment.

3. Real-Time Personalization - Adaptive Learning Platform Adjusting Quiz Difficulty

Scenario

An online learning platform adjusts the difficulty of quiz questions based on the learner's performance during the session.

Use of AWS Streaming Analytics

- ✓ Student Responses stream into Kinesis Data Streams
- ✓ Kinesis Data Analytics computes accuracy/speed metrics
- ✓ Lambda updates the learner's difficulty level and personalizes the next question.

Justification

Personalization occurs instantly based on live user data, making the learning experience dynamic and tailored in real-time.

4. Real-Time Dashboards - Activity & Health Metrics from Smartwatches

Scenario

Fitness trackers collect heart rate, steps, sleep quality and movement data, updating a live dashboard on the user's phone.

Use of AWS Streaming Analytics

- ✓ Device data → Kinesis Data Firehose → S3
- ✓ Data is queried via Athena and ~~visual~~ visualized in QuickSight.
- ✓ Dashboards refresh frequently to display current fitness metrics

Justification

This is a real-time dashboard because users see continuously updated health metrics without delay.

5. Real-Time Anomaly Detection - Smart City Traffic Incident Detection Scenario

Traffic departments analyze live CCTV and signal data to detect accidents, congestion spikes, and unusual road activity.

Use of AWS Streaming Analytics

- Video metadata and signal data → Kinesis Data Streams
- Managed Service for Apache Flink identifies anomalies (e.g., sudden stoppage, density spikes.)
- Lambda → SNS sends alerts to the traffic control center
↳ (Simple Notification Service)

Justification

The system detects anomalies within seconds, enabling immediate intervention - the core requirement of real-time anomaly detection.

Q2 Challenges in Implementing Streaming Analytics and AWS-Based Mitigations

Solⁿ Challenge 1 - Heterogeneous Data Sources and Formats

Organizations ingest real-time data from very different producers - IoT sensors (JSON), application/web logs (text), clickstreams (JSON), database change events and security telemetry. These sources have different schemas, granularities and arrival rates. Building a single stream pipeline that can ingest, parse, normalize and transform all this data in real time is difficult.

AWS Mitigation (Services & How They Help)

✓ Amazon Kinesis Data Streams provides a unified, scalable ingestion layer for all types of producers.

✓ AWS Glue streaming ETL can automatically read multiple data formats (JSON/CSV/Parquet/Avro, etc.), handle schema evolution, and reduce the need for custom parsing.

✓ AWS Lambda can perform lightweight per-record transformations or enrichments before the data flows into storage or analytics systems.

Together, these services enable a single logical streaming pipeline that can process diverse formats and sources without writing custom parsing logic for each one.

② Contd.

Challenge 2 - Choosing Appropriate Delivery Semantics (At-least-Once vs Exactly-Once)

Different business use cases require different processing guarantees.

- ✧ For logs, telemetry, and general clickstreams, duplicates are acceptable (at-least-once)
- ✧ For banking transactions, billing events, and inventory updates, duplicates can cause serious inconsistencies; these require exactly-once processing.

Manually implementing delivery semantics - checkpointing, replay handling, deduplication, and state consistency - is complex and error-prone.

AWS Mitigation (Services & How They Help)

- ✧ Amazon Managed Service for Apache Flink provides exactly-once stateful stream processing through built-in checkpoints and snapshots.
- ✧ Kinesis Data Streams naturally supports at-least-once delivery for non-critical workloads.
- ✧ AWS Lambda consumers can be designed to be idempotent (e.g., using unique keys for upserts) so occasional duplicates do not corrupt downstream data.

This allows architects to match the correct delivery guarantee to the business need without building complex reliability mechanisms themselves.

② Contd.

Challenge 3 - High Throughput and Sudden Bursts in Streaming Traffic

Many streaming workloads must handle rapid spikes and very high event rates. IoT fleets may generate surges during faults, marketing campaigns can create a flood of click events, and application logs may spike during system incidents. Traditional systems struggle to scale horizontally, maintain low latency and prevent data loss under such conditions.

AWS Mitigation (Services & How They Help)

- ✓ Amazon Kinesis Data Streams uses shard-based horizontal scaling, allowing ingestion capacity to grow by simply adding more shards.
- ✓ Enhanced fan-out guarantees dedicated read throughput per consumer, avoiding contention.
- ✓ Auto Scaling Through CloudWatch metrics increases or decreases shard count automatically throughput changes.
- ✓ Amazon Kinesis Data Firehose provides fully managed ingestion that automatically scales buffering and delivery to destinations like S3, Redshift and OpenSearch.

These capabilities provide an elastic, resilient, high-throughput streaming backbone that can absorb spikes without increasing latency or losing data.

Challenge 4 - Shortage of Specialized Streaming Skills and Operational Overhead

Operating custom streaming ecosystems - Kafka clusters, Spark/Flink jobs, custom ETL, scheduling, checkpointing, scaling, patching and recovery - requires specialized distributed systems expertise. Many organisations lack this skill set or cannot allocate dedicated teams for infrastructure management, which slows down or prevents adoption of streaming analytics.

AWS Mitigation (Service & How They Help)

✓ Serverless and managed services such as Kinesis Data Streams, Kinesis Data Firehose, AWS Lambda and AWS Glue streaming eliminate the need to manage servers, clusters or scaling.

✓ Amazon Managed Service for Apache Flink Studio enables SQL/Python/Notebook-based development of streaming applications with minimum boilerplate, lowering the skills barrier.

By offloading infrastructure, scaling and fault-tolerance to AWS, organizations can adopt streaming analytics with far less specialized labour, allowing teams to focus on business logic and insights.

Q3 Generic Vendor-Neutral Streaming Data Architecture

Solⁿ (a) Choice of Architecture

The AWS whitepaper scenarios all follow an Event Stream Processing Architecture!

- ✓ Continuous event ingestion from many producers.
- ✓ A central stream/broker for transport & buffering.
- ✓ Real-time processing (enrichment, aggregation, ML)
- ✓ Storage for later analytics.
- ✓ Delivery to dashboards, alerts and applications.

So, I design a vendor-neutral five-tier streaming architecture based on this pattern.

(b) Five-Tier Vendor-Neutral ArchitectureTier 1: Data Acquisition Layer (Producers)

Role: Generate events and send them into the platform

Typical sub-components:

- ✓ Application front-ends (web, mobile apps)
- ✓ IoT devices and sensors
- ✓ Server/application logs
- ✓ Clickstream trackers (JavaScript tags, SDKs)
- ✓ Database change-data-capture (CDC) connectors
- ✓ Third-party APIs/partner systems.

③ Contd.

Tier 2: Ingestion & Transport Layer (Message Broker)Role: Reliably ingest and transport event streams at scaleTypical sub-components:

- ✓ Ingestion endpoints / gateways (REST/gRPC endpoints, agents)
- ✓ Message broker / event bus
 - eg. a generic distributed log or queue system
- ✓ Partitions / topics / shards for scalability and ordering
- ✓ Buffering & replay support (retention window, offsets)

Tier 3: Stream Processing LayerRole: Perform real-time computations on incoming eventsTypical sub-components

- ✓ Stream processing engine
 - Implements filtering, parsing, joins, enrichment
 - Sliding / tumbling windows, aggregations
 - Pattern detection and anomaly detection
- ✓ State management / state store
 - Maintains counters, windows, session state
- ✓ ML Inference integration
 - Calling external model endpoints or embedded models.

③ Contd.

Tier 4: Storage layer

Role: Persist raw and processed data for query, analytics & history.

Typical Sub-components:

✓ Hot Storage (for recent, query-intensive data)

→ Time-series DB, search index, operational NoSQL store

✓ Warm/Cold storage (for historical data)

→ Data lake/object storage

✓ Analytical warehouse

→ Columnar store for BI and reporting

✓ Metadata/Catalog

→ Schemas, tables definitions, partitions info.

Tier 5: Delivery/Severing layer

Role: Deliver insights, decisions and data products to end users & systems

Typical Sub-Components:

✓ Dashboards & BI tools

→ Operational dashboards, monitoring views.

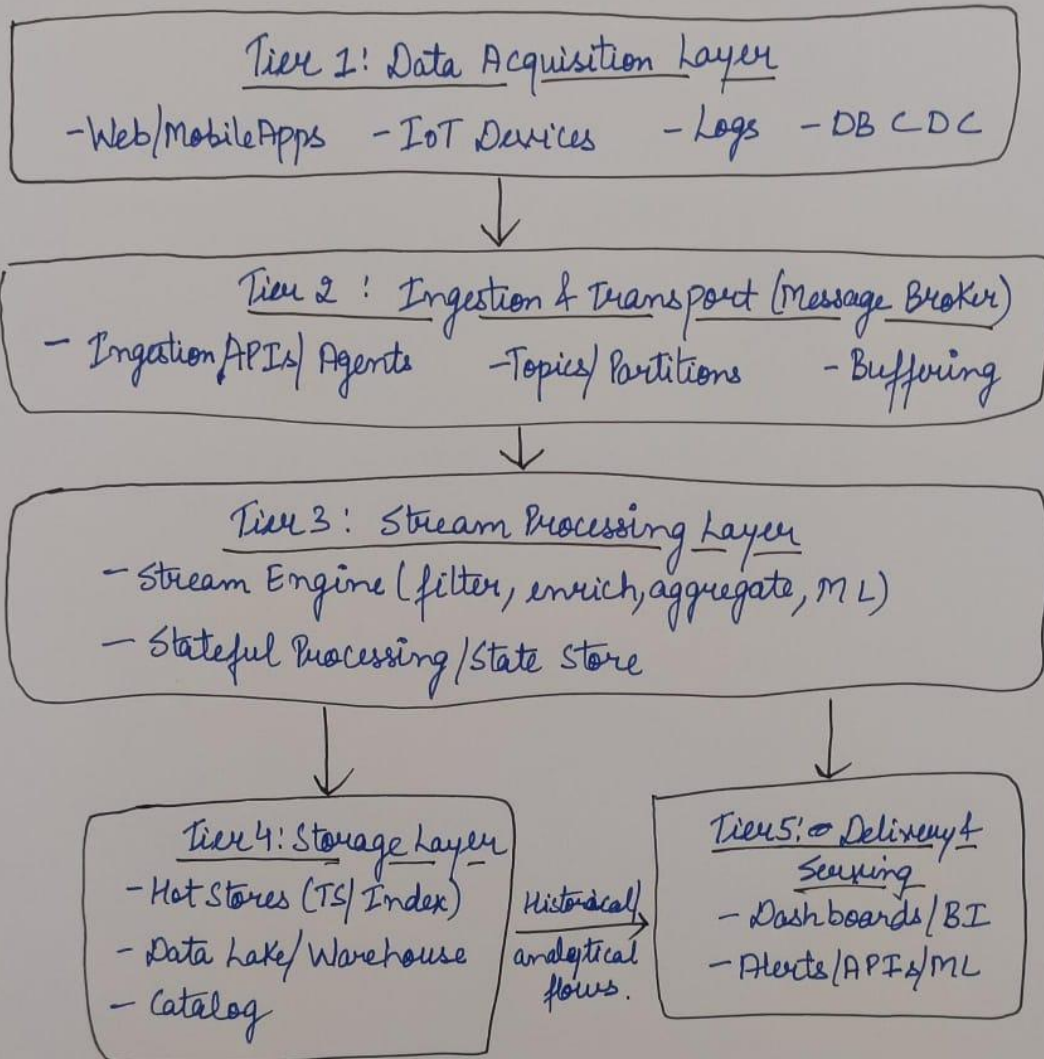
✓ Alerts & notifications → Email/SMS/push/incident alerts

✓ APIs/microservices → Exposed processed data & decisions to applications

✓ Downstream batch/ML pipelines → Model retraining, offline analytics.

(3) Contd.

(C) Vendor-Neutral Block Diagram
(High-level Tiers with Sub-components)



Tier 3 sends real-time insights directly to Tier 5, while Tier 4 supplies historical and analytical data to Tier 5 for BI dashboards, trend analysis and ML workflows

③ Contd.

(d) Mapping to the Three Main Use Cases in the WhitepaperUse Case 1 :- Real-Time Enrichment of User Data for Internet PlansGoal :- Enrich user signup data with bandwidth options in real time based on location.Tier 1 (Data Acquisition) → User signs up via web/mobile; app generates "signup + location" events.Tier 2 (Ingestion & Transport) → Events are sent to the message broker as "user-signup" stream.Tier 3 (Stream Processing) → The stream processor joins signup events with reference data (coverage, plan catalog) in its state store.

→ It computes suitable bandwidth options for the location & user type

Tier 4 (Storage) → Enriched user profiles are written to an operational store or hot storage.

→ Raw events may also be archived in the data lake for later analysis

Tier 5 (Delivery/Serving) → The enriched options are returned to the application/API in near real time.

→ Dashboards can show live signups per region and chosen plan types.

(3) Contd.

Use Case 2: Near-Real-Time Data for Security Teams (Badges at Events)

Goal: Provide aggregated headcount and crowd density every few minutes to security teams

→ Tier 1 (Data Acquisition) → Badge readers and sensors generate entry/exit events as people move across the campus.

→ Tier 2 (Ingestion & Transport) → Agents on servers push transaction logs or sensor events into the message broker.
→ The broker batches & transports these events as a continuous stream.

→ Tier 3 (Stream Processing) → A streaming job groups events by area/zone and time window (e.g., 5 minutes)
→ It computes counts & density metrics per location.

→ Tier 4 (Storage) → Aggregated results are written into an analytical store/warehouse table.

→ Raw badge events are archived in the data lake for historical pattern analysis.

→ Tier 5 (Delivery/Security) → BI dashboards for the security team query the warehouse and refresh every few minutes.

→ Security officers see which zones are crowded and can adjust staffing.

③ Contd.

Use Case 3: Clickstream + ML for Real-Time Price Adjustments (Fast Sneakers)

Goal: Adjust product prices dynamically in near real time based on demand and behaviour.

✓ Tier 1 (Data Acquisition) → Website and mobile apps emit clickstream events (views, add-to-cart, purchases).

→ Product catalog and order updates are also generated as events or change logs.

✓ Tier 2 (Ingestion & Transport) → All these events are sent into the message broker as multiple topics / streams (e.g. "clickstream", "orders", etc.)

✓ Tier 3 (Stream Processing) → The stream processing engine joins clickstreams with product and order data.

→ It builds real-time demand signals (e.g. high view rate + low purchase ^{rate})

→ It calls an ML model (via an inference API) to recommend price adjustments.

✓ Tier 4 (Storage) → Processed features and ML decisions are stored in hot storage and in the data lake.

→ Updated price recommendations are stored in an operational product store.

✓ Tier 5 (Delivery/Severing) → The e-commerce application reads the latest prices from the severing store and displays them to users.

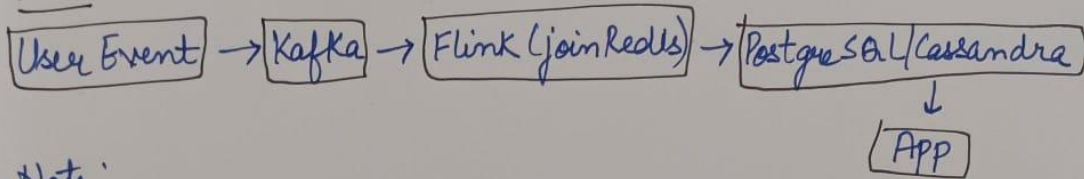
→ Dashboards for business teams show live performance of price changes.

Q4 Open-Source Alternatives for AWS Streaming Use Cases

Scenario 1 - Real-Time Internet Plan RecommendationOSS Stack:

Kafka (ingestion), Flink/Kafka Streams (real-time join), Redis (lookup store), PostgreSQL/Cassandra (operational DB), Superset (dashboard)

Reasoning → Kafka provides durable ingestions; Flink/Kafka Streams support low-latency enrichment using Redis for location-based lookups. The enriched result is written to a fast operational store for the application.

FlowNote:

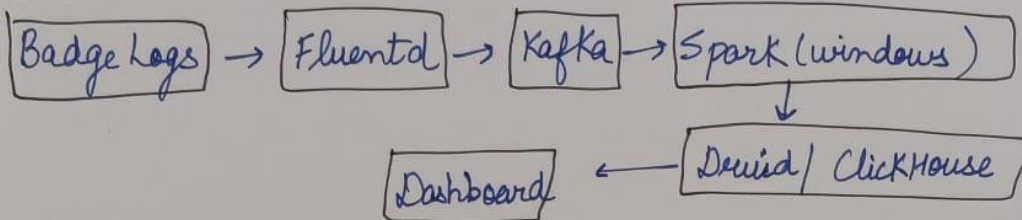
Equivalent to Kinesis + Lambda but requires self-managed scaling and failover.

Scenario 2 - Security Analytics (Near-Real-Time counts)OSS Stack:

Fluentd/Filebeat → Kafka → Spark Structured Streaming/Kafka Connect
→ Druid/Click House → Grafana/Superset

Reasoning:- Agents forward logs into Kafka, Spark performs windowed aggregations (e.g., 5-min counts). Druid/Click House support fast OLAP queries for dashboards.

(4) Contd.

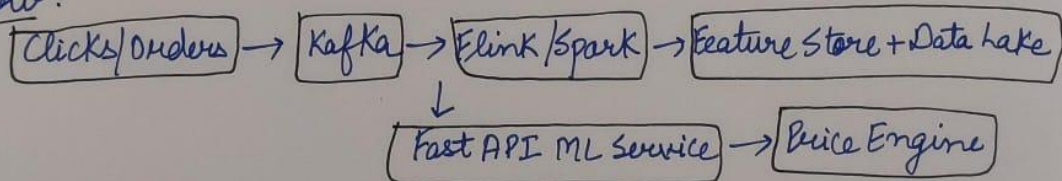
Flow:

Note: Replicates Firehose → Redshift, but ingestion tuning and OLAP indexing must be handled manually.

Scenario 3 - Dynamic Pricing with Clickstream + ML

OSS stack: Kafka (clicks/orders), Flink/Spark streaming (joins/features), Redis/Cassandra (feature store), Hudi/Delta Lake (historical), ML flow (training), Fast API (online inference), Superset (analytics)

Reasoning: Kafka captures high-volume events. Flink/Spark derives demand features. Feature store enables low-latency inference. Hudi/Delta Lake supports incremental data for ML retraining.

Flow:

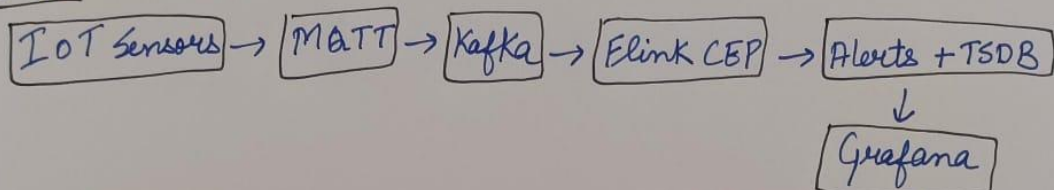
Note: Comparable to Kinesis + Glue + DynamoDB + SageMaker but requires manual cluster and model lifecycle management.

④ Contd.

Scenario 4 - IoT Sensor Anomaly DetectionOSS Stack:

MATT ~~broker~~ → Kafka → Elink CEP → Timescale DB / InfluxDB
 ↓
 Grafana ← Alert Service

Reasoning: MATT handles device messages. Kafka buffers them. Elink CEP detects anomaly patterns in real-time. Time-series DB stores readings for visualization.

Flow:

Note: Matches AWS IoT + Kinesis + Elink, but IoT connectivity and throughput scaling are manually administered.

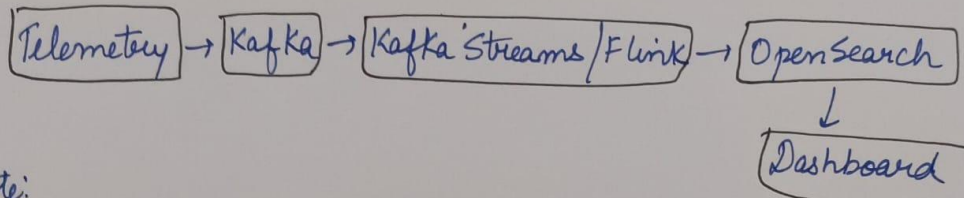
Scenario 5 - Telemetry Monitoring PipelineOSS Stack:

~~Kafka~~ → ~~Kafka Streams / Elink~~ → ~~OpenSearch~~ → ~~Dashboards~~
 Kafka → Kafka Streams / Elink → OpenSearch / ElasticSearch
 ↓
 Grafana / Kibana

(4) Contd.

Reasoning: Kafka remains the core ingestion backbone. Kafka Streams/Flink normalize telemetry. OpenSearch indexes logs for querying.

Flow:



Note:

Similar to MSK-based pipelines but requires internal management of partitions, brokers and index performance

Why These OSS Architectures Are Equivalent?

All solutions preserve the AWS pattern of ingestion → processing → storage → serving, but use self-managed open source components.

Functionally, the workflows remain the same, while operations (scaling, failover, storage tuning) shift to the engineering team.